

■ CRA TEAM 3 · CTRL-C

CodeFab

새로운 확장자 `.ctrlc` 로 코드를 분석하고 실행하는
Interpreter

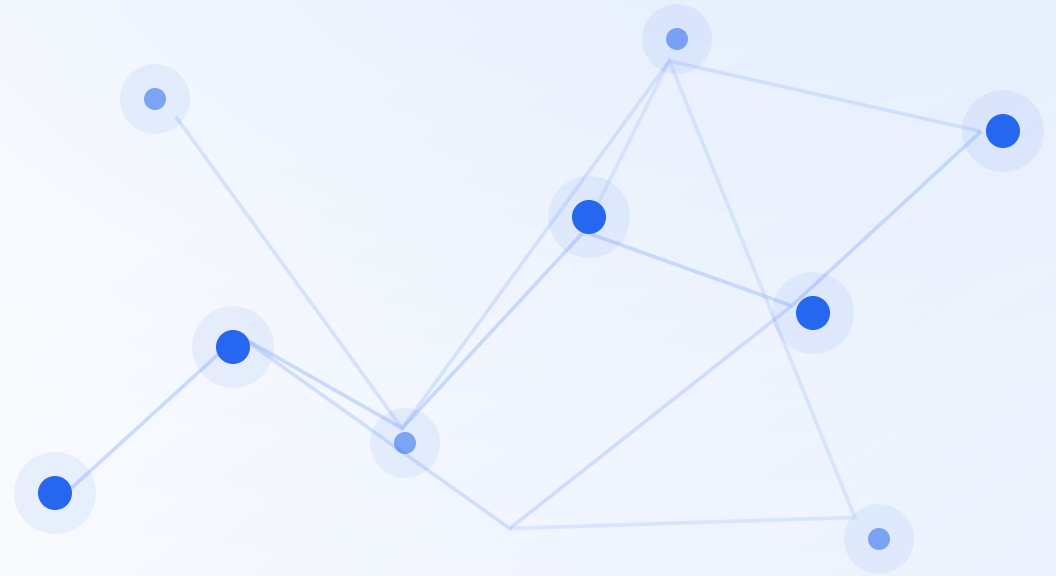
Language

Pipeline

Shell

Design Patterns

Collaboration



목차

01

팀 소개

역할과 구현 분담

02

프로젝트 목표

새로운 확장자 `.ctrlc`

03

전체 구조

Shell과 interpreter pipeline

04

주요 기능

Tokenizer · AST · Runtime

05

디자인 패턴

AST · Callable · Command

06

협업과 품질

Review · TDD · Actions

■ TEAM

팀 역할과 구현 분담

김명준 팀장

architecture · CI

권소영

assembler · executor

김민준

expression · shell

박진우

tokenizer · array

이예진

checker · optimizer

`.ctrlc` 실행 목표

입력

- `.ctrlc` source
- prompt 한 줄 입력
- debug 실행 대상



결과

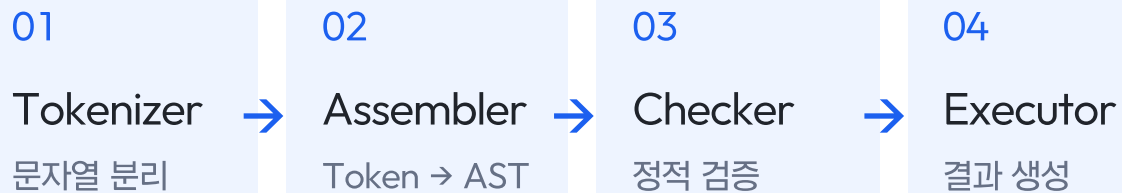
- Token / Expr / Stmt 구조화
- Checker 사전 검증
- Executor 실행 결과 생성

Shell과 Pipeline 구조

Shell Modes

- 01 Prompt Mode
한 줄 입력을 즉시 실행
- 02 File Mode
.ctrlc 파일 전체 실행
- 03 Debug Mode
Stmt 단위로 실행 상태 확인

Interpreter Pipeline



■ FEATURE IMPLEMENTATION

기능 구현 범위

각 기능은 입력, 내부 구조, 실행 결과 기준으로 설명합니다.

01

Parse

문법 해석

Tokenizer · AST

Runtime

실행 컨텍스트

Environment · Execute

Object

함수 / Class

Callable · This · Super

Module

Array · Import

데이터 · 파일 재사용

Safety & Shell

Checker · Shell

검증 · 실행 모드

문자열은 **Tokenizer**를 지나며 의미 단위로 쪼개집니다

Tokenizer

소스 코드 문자열을 한 글자씩 훑으면서 예약어, 식별자, 연산자, 리터럴 단위의 token으로 분리합니다.

01 공백 스킵

공백은 건너뛰고 개행마다 line을 +1합니다.

02 예약어 · 식별자 구분

예약어 표와 identifier 가능 여부를 확인합니다.

03 Token 저장

TokenType과 origin을 저장해 Assembler로 전달합니다.

var [] name [] = [] "ctrlc" ;

↓ 한 글자씩 확인하며 종류에 따라 분기

var 문자로 시작 → 예약어 테이블 조회 → 일치

name 문자로 시작 → 예약어 테이블 조회 → 없음

= 기호로 시작 → 등록된 토큰값과 일치

"ctrlc" "로 시작 → 닫는 "가 나올 때까지 읽어 하나로 묶음

; 기호로 시작 → 등록된 토큰값과 일치

↓ Token(type, origin)으로 저장

VAR
var

IDENTIFIER
name

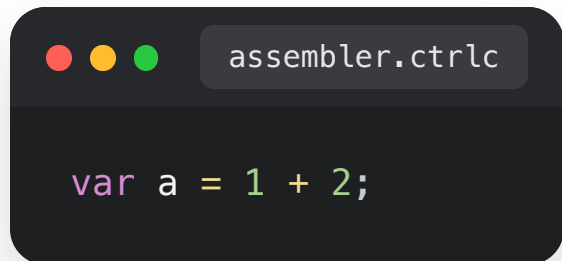
EQUAL
=

STRING
"ctrlc"

SEMICOLON
;

이 문장에서 나온 Token은 5개 · 전체 TokenType은 총 44개 정의되어 있습니다.

Assembler는 Token을 실행 가능한 AST로 조립합니다



생성된 AST

VarStmt

```
└─ BinaryExpr(+)
   └─ LiteralExpr(1)
      └─ LiteralExpr(2)
```

Assembler는 실행하지 않고
AST만 생성합니다.

Expression

값을 계산하는 node

Literal · Binary · Variable · Call · Get

Statement

실행 흐름을 표현하는 node

Var · Print · Block · If · For · Func · Class · Import

Environment는 식별자와 Scope Chain을 관리합니다

Global Environment

변수

x = 10

Native Function

print · Array

User Function

closure

Class

method table

Instance

field · method

Import

alias · member

Function 호출 시 Scope Chain

Function Environment

a = 2

b = 3

local variables

parent
→

Global Environment

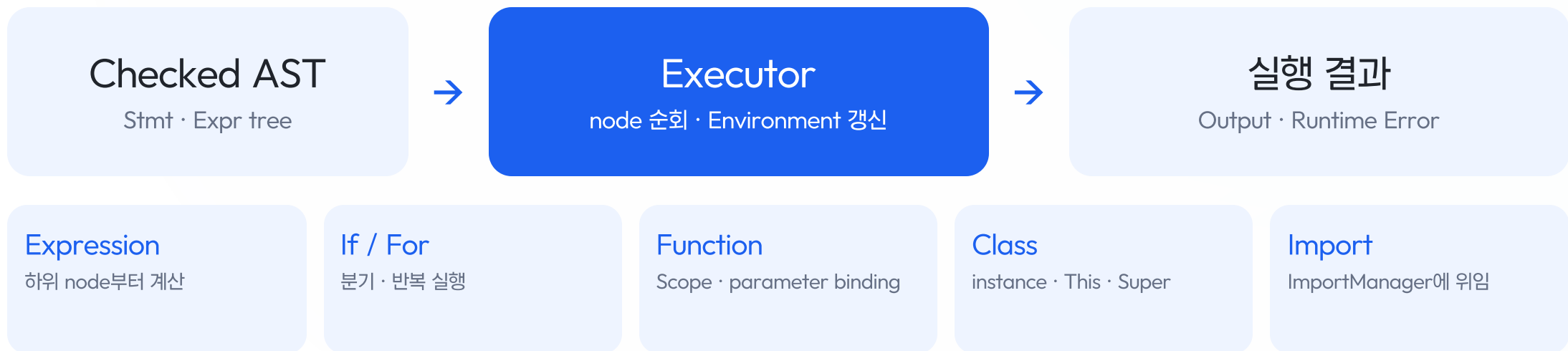
functions

classes

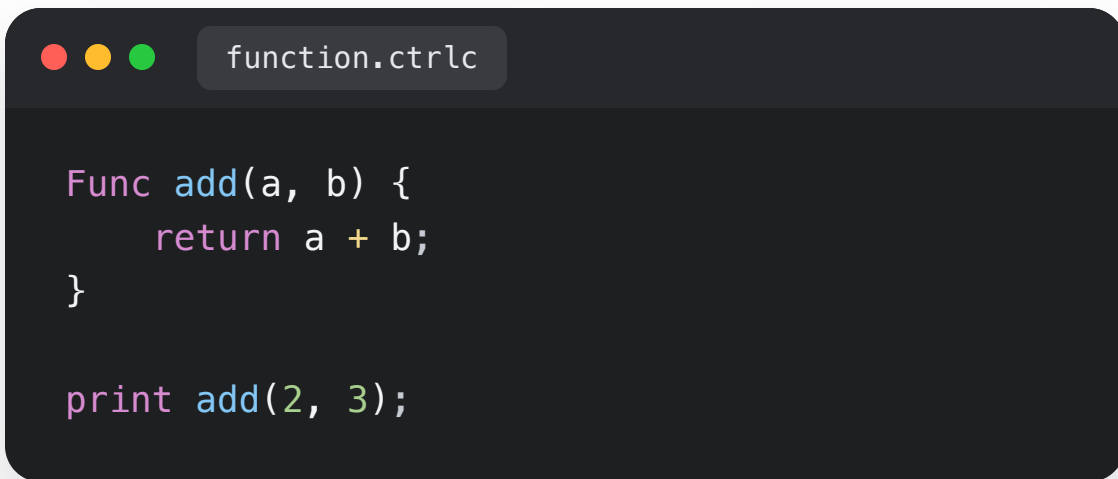
imports

현재 Scope에 이름이 없으면 parent를 따라 Global까지 탐색합니다.

Executor는 AST를 순회하며 실제 프로그램을 실행합니다



Function은 새 Environment에서 body를 실행합니다



```
function.ctrlc

Func add(a, b) {
  return a + b;
}

print add(2, 3);
```

호출 흐름

선언 시 함수 객체를 저장하고, 호출 시 새 Environment에서 body를 실행합니다.

- 01 선언
FunctionStmt → UserFunction
- 02 호출
parameter와 argument binding
- 03 반환
return 값을 호출 지점으로 전달

Class는 method table과 This binding으로 동작합니다

객체 실행 흐름

class 생성과 method binding을 Callable 흐름으로 처리합니다.

- 01 UserClass
method table
- 02 UserInstance
field / method lookup
- 03 This binding
instance 연결
- 04 Super lookup
parent method 탐색

```
class.ctrlc

Class SpeedRobot : Robot {
    move() {
        Super.move();
        print "speed";
    }
}

var r = SpeedRobot();
r.move();
```

Array() 로 배열을 생성할 수 있습니다

생성과 접근

Function과 Class가 사용하는 Callable 규칙으로 Array를 생성합니다.

01 Array() 호출

NativeFunction 등록 · Callable.call()로 생성

02 크기 검사 후 생성

0 이상의 정수 확인 · [None] * size 초기화

03 배열 사용

a[i] → IndexGetExpr / IndexSetExpr

04 실행 중 검증

index 정수 여부와 범위 검사

```
var arr = Array(3);  
arr[0] = "first";  
  
print arr[0];
```

ImportManager가 Runtime Import를 다시 파싱합니다

Executor

ImportStmt 감지 · Checker 실행 · alias namespace 등록

ImportManager

경로 해석 · cache · 순환 추적 · Tokenizer와 Assembler 호출

ImportStmt
Executor가 감지



File Read
.ctrlc source



Tokenizer · Assembler
source → AST



Checker
import 대상 검증



Module Env
alias namespace 등록

Checker

중복 import · alias 충돌

ImportManager · Assembler

순환 import · 반복문 내부 import · 선언 외 구문

Checker 실행 전 검증

ConstantFolder

계산 가능한 expression을 LiteralExpr로 치환

ScopeStack

선언, 초기화, scope distance 추적

Resolver

This, Super, return, import 규칙 확인

Scope

중복 선언 · 미초기화 변수

Class

This/Super · 자기 상속

Import

중복 import · 순환 import

Report

source line 기준 오류

Static Binding

01. 변수 사용

`VariableExpr(name)`

어떤 scope의 선언을 가리키는지 아직 결정되지 않은 상태입니다.



02. Checker

scope depth 계산

ScopeStack에서 선언 위치를 찾아 distance를 계산합니다.



03. AST 기록

`expr.distance = depth`

계산한 거리를 Expr에 저장해 실행 시 다시 탐색하지 않습니다.



04. Executor

`get_at(distance)`

정해진 상위 Environment로 바로 이동해 값을 읽습니다.

Shell은 실행 모드를 고르고 Pipeline으로 위임합니다

실행 모드

- 01 Prompt Mode
한 줄 입력을 즉시 실행
- 02 File Mode
.ctrlc 파일 전체 실행
- 03 Debug Mode
breakpoint, step, watch로 상태 확인

Shell 책임

입력 분류 · command / code 판단

상태 관리 · history / debug context

Factory → Command.execute(shell)

Runner → 공통 Interpreter Pipeline

Custom Language Features

확장 방향

팀 전용 문법도 같은 pipeline에서 처리되도록 확장했습니다.

한글 alias

keyword token으로 정규화

`ctrl_c()`

팀 소개 native function

♡ 연산

문자열 기반 점수 계산

`.ctrlc`

file mode와 import 기준

02

■ DESIGN PATTERNS

디자인 패턴

복잡해지는 구현 지점을 패턴으로 어떻게 정리했는지 봅니다.

패턴 적용 위치

AST 구조

Composite-style AST

Stmt와 Expr node가 tree로
중첩되고 Checker와 Executor가 같은
구조를 순회합니다.

호출 규칙

Callable Adapter

Native Function, User Function,
Class 생성을 같은 call 계약으로
통일합니다.

Shell 명령

Command Pattern

prompt 입력별 동작과 history 정책을
독립 Command 객체에 둡니다.

Composite-style AST로 문법 구조와 실행 구조를 공유합니다

입력 문법

```
for (var i = 0; i < 3; ...)
```

Assembler가 네 요소를 분리합니다.



ForStmt

VarStmt

BinaryExpr

AssignExpr

BlockStmt



같은 AST 사용

Checker

Executor

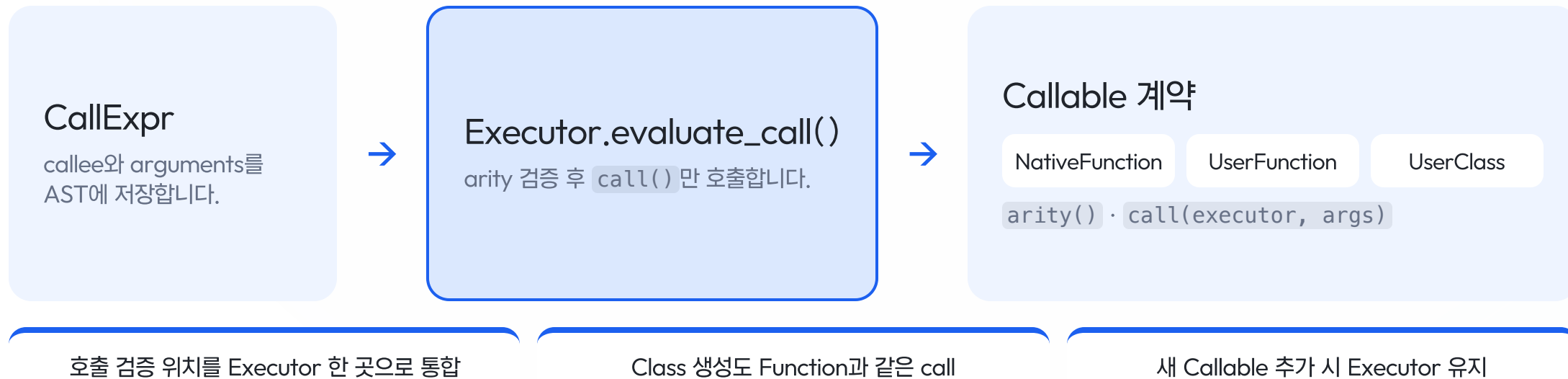
검증과 실행이 동일한 node tree를 순회합니다.

Assembler가 AST를 한 번만 생성

Checker와 Executor 구조 중복 제거

문법 추가 시 node 단위로 확장

Callable Adapter로 모든 호출을 하나의 계약에 맞춥니다



Command Pattern은 Shell 명령을 독립 실행 단위로 분리합니다



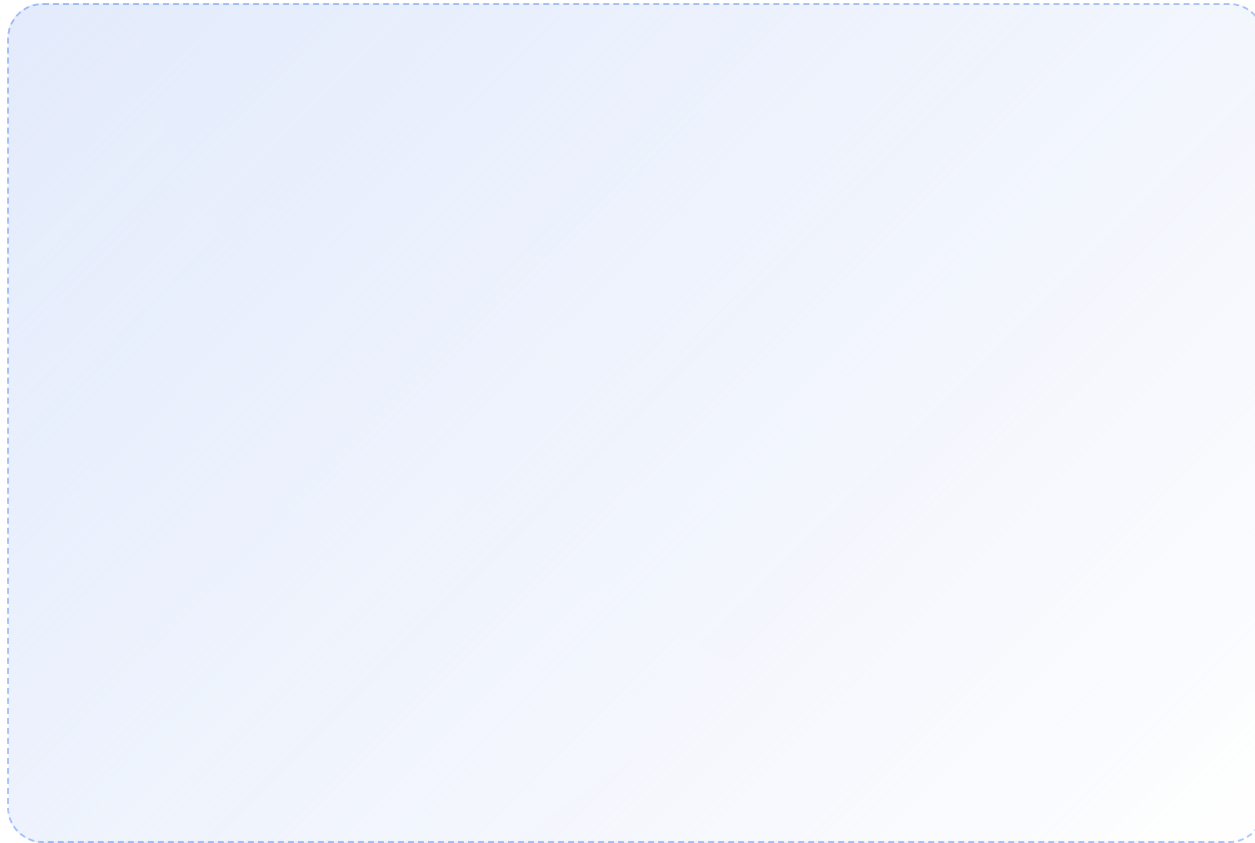
03

■ COLLABORATION

협업과 품질 기준

리뷰, 자동화, 보호 규칙으로 main 반영 기준을 맞췄습니다.

Code Review 사례



의도 공유

변경 이유와 검증 범위 기록

관점 확보

최소 2명 approve

수정 근거

리뷰 의견으로 누락 보완

TDD와 회귀 테스트 사례

실패 고정

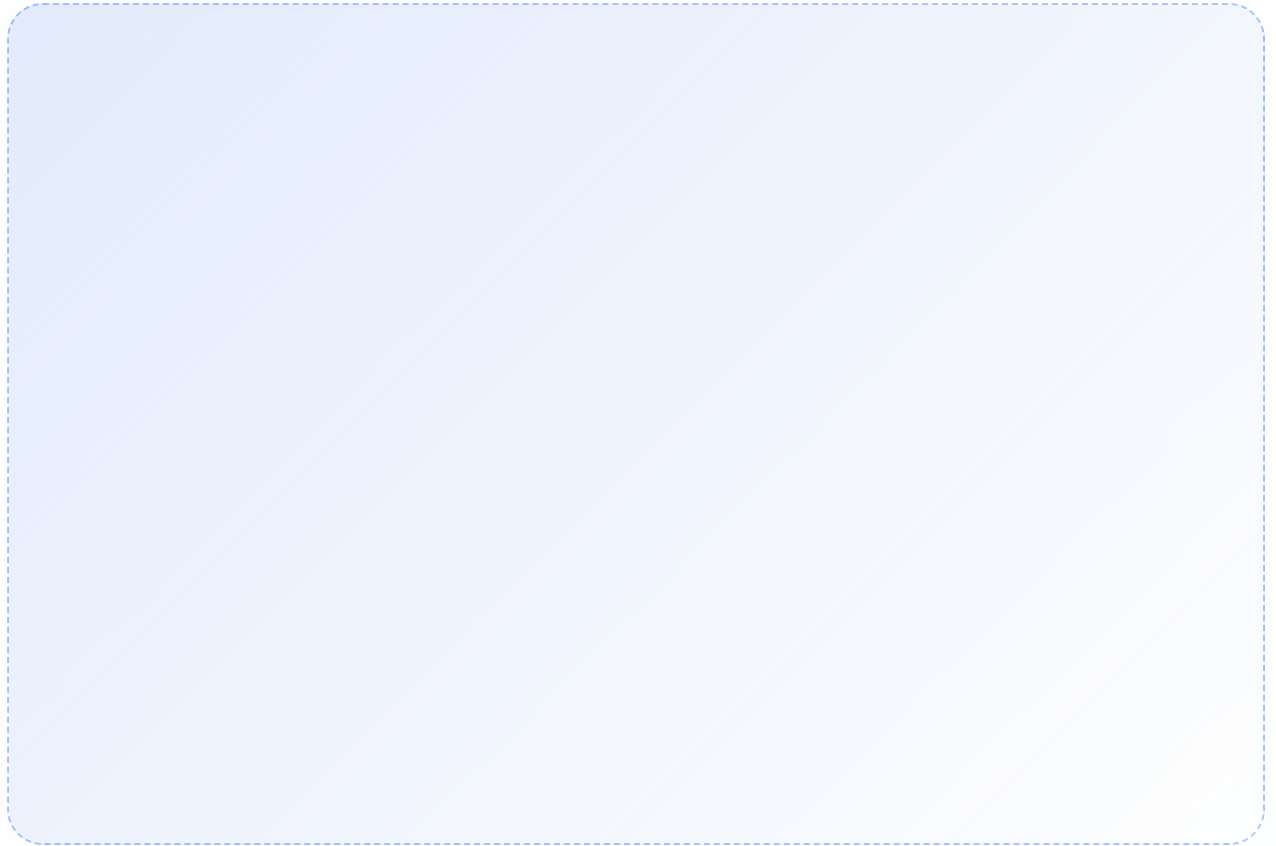
구현 전에 깨져야 하는 케이스를 먼저 남겼습니다.

회귀 방지

import, debug, checker 오류를 테스트로 관리했습니다.

자동 검증

Actions가 PR마다 같은 기준으로 실행했습니다.



Shell Command Refactoring

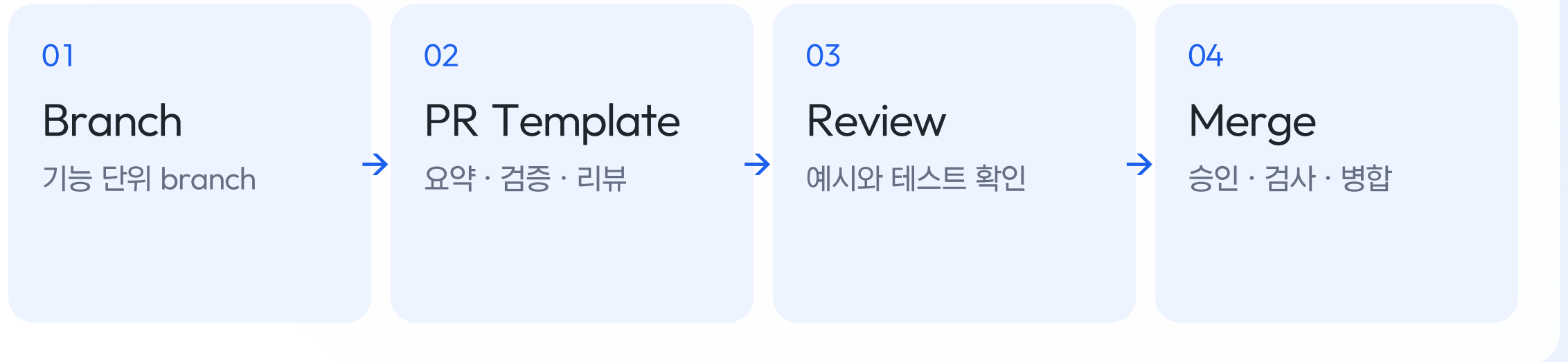
Before

- PromptShell 안에서 명령 분기가 커짐
- 기능 추가 시 기존 흐름을 계속 건드림
- 명령별 테스트 범위가 넓어짐

After

- 명령을 Command 객체로 분리
- PromptShell은 command 선택과 실행만 담당
- 새 shell 기능을 독립 단위로 추가

PR 작성과 검토 흐름



Actions와 Protection Rule

GitHub Actions

- black, isort, ruff로 코드 스타일 검증
- pytest와 coverage report 실행
- gist acceptance로 대표 언어 흐름 확인

Protection Rule

- main 직접 반영 방지
- approve 후 merge
- required check 통과 후 반영